AIX

Robotics (1)

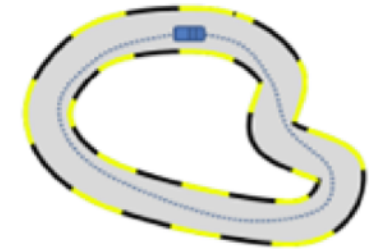
Mingyu Kim,
June, 5th, 2026

Why Imitation Became Attractive

- Early robot learning used human demonstrations because trial-and-error learning was expensive and slow.
- The historical goal was simple: convert expert behavior into a policy that can act autonomously.

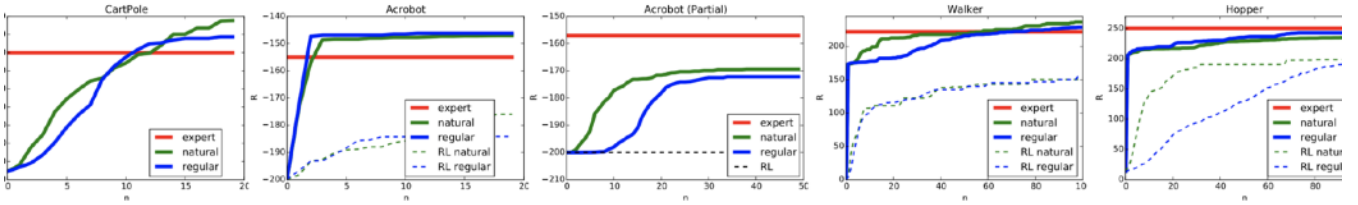
Autonomous Driving as an Imitation Task

- Driving made imitation learning intuitive: camera observations can be paired with human steering actions.
- This turns control into a supervised mapping from observation to action.



Why Not Learn Everything with Reinforcement Learning?

- Pure reinforcement learning can require too many real-world trials for physical robots.
- Imitation learning starts from expert demonstrations, which makes early learning far more sample-efficient.



(a) Cartpole (b) Acrobot (c) Acrobot (POMDP) (d) Walker (e) Hopper

ure 2. Performance (cumulative reward R on y-axis) versus number of episodes (n on x-axis) of AggreVaTeD (blue and green erts (red), and RL algorithms (dotted) on different robotics simulators.

Imitation learning is **exponentially lower sample complexity** than Reinforcement Learning for sequential predictions

Imitation Is Not Ordinary Supervised Learning

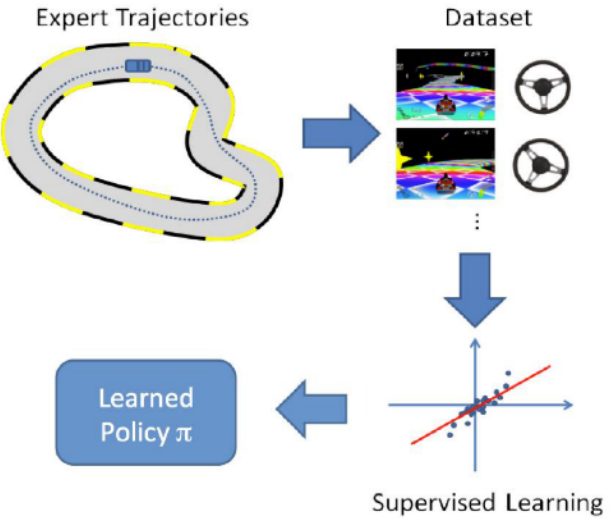
- In standard supervised learning, predictions do not change the future input distribution.
- In robotics, every action changes the next state, so mistakes reshape the data the policy will see.

Imitation Learning:

- Predictions lead to actions that will change the world and affect future actions
  - Data is highly correlated

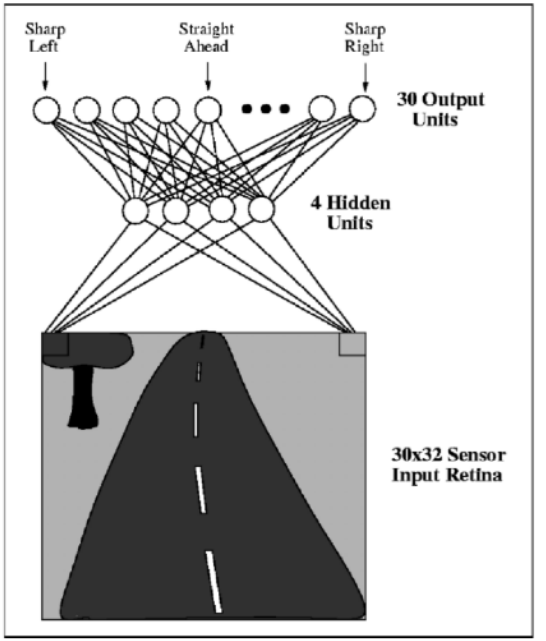
Behavioral Cloning: Expert Trajectories to Policy

- Behavioral cloning collects expert trajectories and trains a policy with supervised learning.
- The learned policy imitates expert actions, but only on states similar to the demonstrations.



What Behavioral Cloning Actually Learns

- The policy learns expert actions on the expert's state distribution.
- It does not automatically learn how to recover from states the expert never visited.



The Post-Mortem: The Data Distribution Was the Problem

- The failure was not simply a weak model or too little data from the same expert distribution.
- The missing data was recovery behavior after the robot drifts away from the expert trajectory.

- Insufficient Model Capacity?

Linear predictor sufficient in imitation learning case

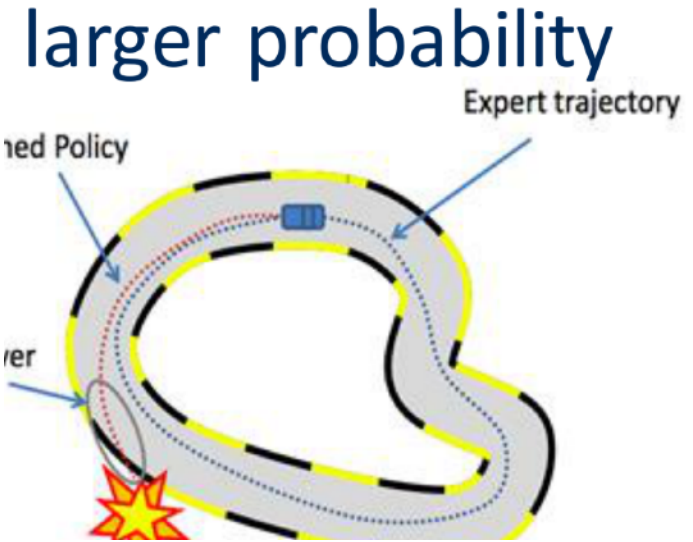
- Too small of a dataset?

Larger training set data does not improve performance

Hold-out errors close to training errors

Small Mistakes Create Cascading Errors

- A small steering error can move the robot into a state absent from the training data.
- Once off-distribution, later predictions become less reliable and errors cascade.



Covariate Shift:
Why Sequential Prediction Is Hard

- Supervised learning assumes train and test examples are drawn from the same distribution.
- Sequential decision-making violates this because the learned policy induces its own state distribution.

From Linear Error to Quadratic Error

- Independent prediction errors scale roughly like $T\varepsilon$ over a horizon of length T.
- Correlated sequential errors can scale like $O(T^2\varepsilon)$, which explains why naïve imitation is fragile.

Supervised Learning =
Independent data points

Error Bound: **$T\varepsilon$ over T decisions**

Structured Prediction
→ Highly correlated data
→Cascading errors

Best expected error: **$O(T^2\varepsilon)$ over T decisior**

DAgger: Dataset Aggregation

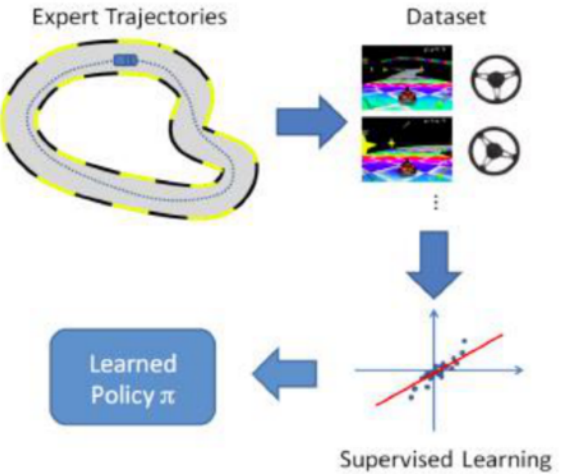
- DAgger fixes covariate shift by collecting data on states visited by the learner, not only the expert.
- The expert relabels those states, so the policy gradually learns recovery behavior.

Initialize $\mathcal{D} \leftarrow \emptyset$.
Initialize $\hat{\pi}_1$ to any policy in Π.
for $i = 1$ **to** N **do**
  Let $\pi_i = \beta_i \pi^* + (1 - \beta_i)\hat{\pi}_i$.
  Sample T-step trajectories using π_i.
  Get dataset $\mathcal{D}_i = \{(s, \pi^*(s))\}$ of visited states by π_i and actions given by expert.
  Aggregate datasets: $\mathcal{D} \leftarrow \mathcal{D} \bigcup \mathcal{D}_i$.
  Train classifier $\hat{\pi}_{i+1}$ on $\mathcal{D}$.
end for
Return best $\hat{\pi}_i$ on validation.

Algorithm 3.1: DAGGER Algorithm.

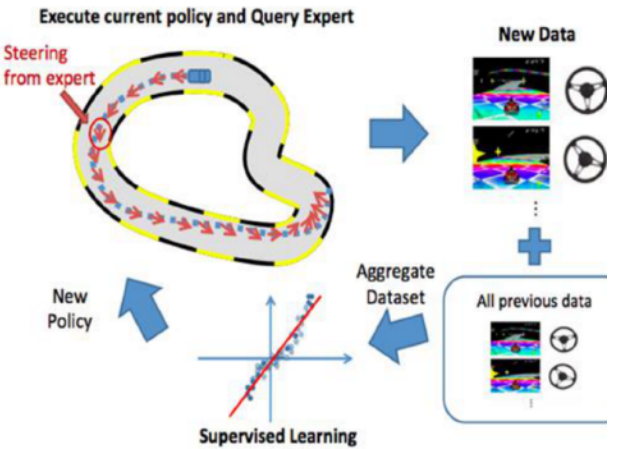
DAgger Step 1:
Start Like Behavioral Cloning

- The first policy is trained from expert demonstrations, exactly like ordinary behavioral cloning.
- This gives a baseline controller before interactive correction begins.



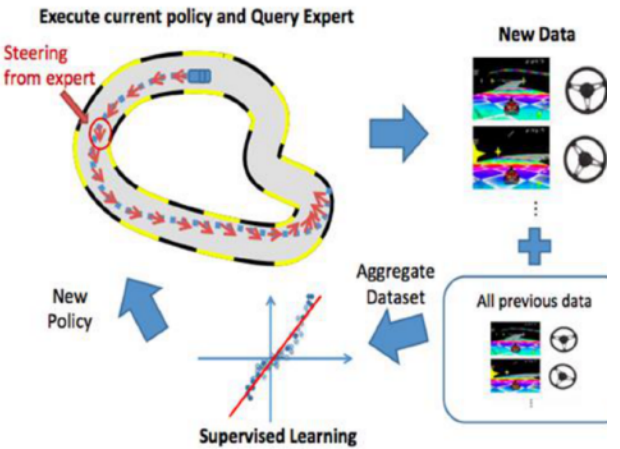
DAgger Step 2:
Roll Out the Learner

- The learner is allowed to act, so the dataset includes the states it actually visits.
- The expert then provides the correct action for those learner-induced states.



DAgger Step 3:
Aggregate and Retrain

- Each iteration adds newly labeled learner states to the growing dataset.
- As β_i decays, the policy becomes trained more directly on its own state distribution.



What DAgger Fixed

- DAgger changed the data-collection process, not just the network architecture.
- Its historical value was reducing sequential error growth by training on on-policy states.

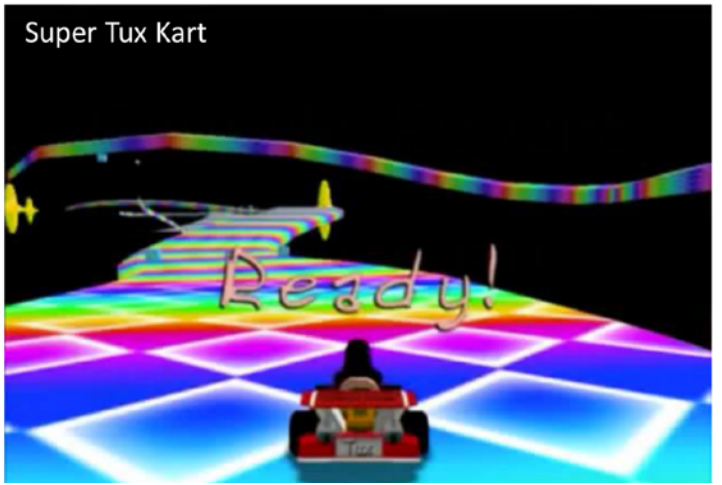
DAgger (Dataset Aggregation) :

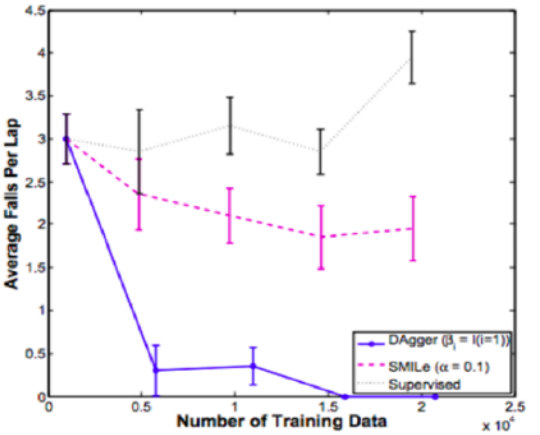
- Uses Interaction
- Have human expert to provid correct execution

Expected error:
$O(T\varepsilon)$ over T decisions
instead of **$O(T^2\varepsilon)$**

DAgger in Game-Like Control

- The SuperTuxKart example shows DAgger improving control through repeated correction.
- This makes the algorithm easy to teach: the learner drives, fails, gets labels, and improves.

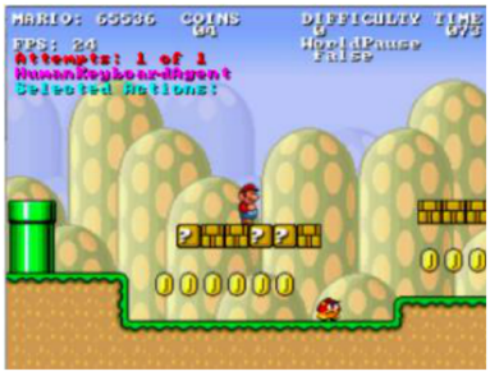


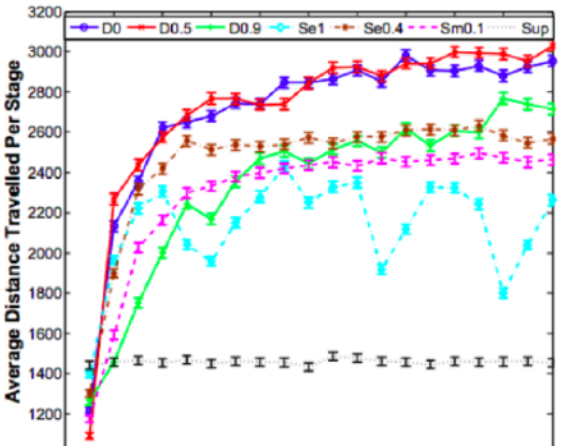


- Correct own mistakes

DAgger Beyond One Domain

- The Super Mario example shows that DAgger is a general sequential prediction idea.
- Any task where actions affect future inputs can suffer from the same covariate-shift problem.





The Remaining Bottleneck: Generalization

- DAgger made task-specific imitation more robust, but it still depended on expert corrections and narrow task data.
- This limitation motivates the modern move toward foundation models, world models, and VLA-style generalization.

Problems:

- Teacher is not truly an optimal controller
- World does not operate as simple Markov Decision Process
- Given a single behavior, there are many cost functions that lead to the same behavior (indeterminate)